CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

School of Business and Management

Master of Business Administration (MBA)

Batch 2025 – 27

Trimester III

PYTHON END TERM PROJECT REPORT

Predictive Lead Scoring & Decision Support System

B2C Analytics Pipeline Using Machine Learning

Submitted by: Alex Abraham

Class: 3 MBA BA PLC

Course Code & Name: MBA 342 B– Python for Data Analysis

Campus of Study: Pune Lavasa Campus

Faculty In-charge: Dr. Himani Saraswat

Dated: 8-05-2025

INDEX

S. No.	Content
1	Executive Summary
2	Abstract
3	Chapter 1: Introduction
4	Chapter 2: Problem Statement & Objectives
5	Chapter 3: Dataset Description
6	Chapter 4: Data Preprocessing
7	Chapter 5: Encoding & Feature Scaling
8	Chapter 6: Feature Engineering -- The Engagement Score
9	Chapter 7: Machine Learning Framework
10	Chapter 8: Model Evaluation & Performance Analysis
11	Chapter 9: Feature Importance Analysis
12	Chapter 10: Decision Support System (DSS) Architecture
13	Chapter 11: Business Impact & ROI Analysis
14	Chapter 12: Technology Stack
15	Chapter 13: Conclusion
16	Chapter 14: Limitations & Future Scope
17	Appendix A: Python Implementation Code for Predictive Lead Scoring & Decision Support System

Executive Summary

This report presents a comprehensive business analytics capstone project centred on Predictive Lead Scoring and Decision Support for X Education, a B2C online education platform. The project addresses a fundamental commercial challenge: the organisation's sales team was allocating effort indiscriminately across a heterogeneous lead pool whose historical conversion rate was only 38.5%, resulting in significant resource inefficiency and foregone revenue.

To resolve this, an end-to-end machine learning pipeline was architected and deployed: commencing with structured data ingestion from a 9,240-row, 37-feature dataset; progressing through rigorous preprocessing, outlier management, encoding, and domain-driven feature engineering; and culminating in the training and evaluation of two supervised classification models—Logistic Regression (baseline) and Random Forest (champion).

The champion Random Forest model, trained on 300 decision trees with class-balancing and stratified sampling, achieved a ROC-AUC of 0.884, an accuracy of 81.3%, and a recall of 81.9%—correctly identifying 862 of the 1,068 converting leads in the test cohort. A proprietary Engagement Score feature, constructed from normalised website behavioural signals, ranked among the top four predictors, validating the hypothesis that digital engagement behaviour is a reliable precursor to purchase intent.

The analytical outputs were operationalised through an interactive Decision Support System (DSS) dashboard, built with ipywidgets and integrated directly with the serialised machine learning model. The DSS enables sales managers to score individual leads in real time, categorise them into strategic tiers (HOT, WARM, COLD, and DEAD), and receive prescriptive sales directives calibrated to each tier, without requiring any data science expertise.

Abstract

Lead conversion prediction represents one of the most commercially impactful applications of supervised machine learning in the B2C context. This project constructs and validates a Predictive Lead Scoring and Decision Support System for X Education, leveraging a dataset of 9,240 leads characterised by 37 behavioural and demographic attributes. The analytical pipeline encompasses systematic data preprocessing—including median and mode imputation, IQR-based outlier clipping, binary and one-hot encoding—followed by the engineering of a composite Engagement Score that aggregates normalised website activity signals into a single predictive construct.

Two classification models were evaluated: Logistic Regression, serving as an interpretable baseline, and a Random Forest ensemble, which emerged as the champion model with a ROC-AUC of 0.884 and an accuracy of 81.3%. The ensemble's superior performance is attributed to its capacity to capture nonlinear feature interactions and high-cardinality categorical patterns that linear models cannot represent. Model outputs were integrated into an ipywidgets-based Decision Support System that delivers real-time lead scores, tier assignments, and prescriptive sales directives. Serialised model artefacts (champion_model.pkl, data_scaler.pkl, model_columns.pkl, norm_stats.pkl) enable deployment without retraining the model.

The system addresses the dual objectives of predictive accuracy and operational usability, providing a scalable, data-driven alternative to intuition-based lead qualification. Future extensions include Streamlit-based web deployment, SHAP explainability integration, and CRM API connectivity.

Chapter 1: Introduction

1.1 Background and Business Context

In contemporary B2C commercial environments, the proliferation of digital marketing channels has fundamentally altered the economics of lead generation. Organisations can now attract thousands of prospective customers through paid search, social media, organic content, and referral networks—but the sheer volume of inbound leads has outpaced the capacity of sales teams to pursue them all with equal diligence. The result is a systematic inefficiency: high-potential leads receive the same delayed or generic follow-up as low-intent enquiries, eroding conversion rates and driving up the cost of customer acquisition.

X Education, an online learning platform operating in the competitive Indian EdTech sector, exemplifies this challenge. Despite a consistent inbound lead flow, the organisation's overall conversion rate languished at 38.5%—meaning that for every ten leads entering the funnel, more than six were never converted, and the sales effort expended on non-converting leads represented a direct operational cost. This context establishes the commercial imperative for a data-driven lead-scoring system capable of distinguishing high-probability converters from low-probability ones at the point of lead entry.

1.2 The Role of Predictive Analytics in Sales

Predictive analytics employs statistical modelling and machine learning to derive forward-looking insights from historical data. In the context of sales, this manifests as lead scoring: assigning a quantitative conversion probability to each prospect based on their observed behavioural and demographic characteristics. Unlike rule-based scoring systems—which rely on static heuristics defined by sales management—machine learning models learn conversion patterns empirically from thousands of historical outcomes, capturing subtle interactions between variables that human intuition cannot reliably encode.

The academic literature consistently demonstrates that ensemble methods such as Random Forests outperform linear classifiers for lead scoring tasks, owing to their capacity to model nonlinear relationships and resist overfitting through bootstrap aggregation. This project operationalises these insights within a full analytical pipeline, from raw data ingestion through to interactive decision support.

1.3 Project Scope and Significance

This project encompasses the design, implementation, evaluation, and deployment of a complete Predictive Lead Scoring and Decision Support System. The scope includes data preprocessing, feature engineering, model training and evaluation, and the construction of an interactive dashboard—constituting an end-to-end analytics solution suitable for production deployment. The significance of this work lies not only in its technical rigour but in its direct business applicability: every component of the system is designed to translate analytical outputs into actionable sales decisions, bridging the gap between data science and commercial execution.

Chapter 2: Problem Statement & Objectives

2.1 Problem Statement

X Education's sales organisation faces a classical precision-recall trade-off in its lead qualification process. The absence of a systematic scoring mechanism compels sales representatives to engage all inbound leads with approximately equal intensity, rendering the function operationally inefficient and strategically suboptimal. The consequences are threefold: first, revenue leakage from high-intent leads that receive delayed follow-up; second, resource wastage on low-intent leads that consume disproportionate sales bandwidth; and third, the inability to scale the sales function without a commensurate increase in headcount.

2.2 Research Objectives

The following objectives govern the design and execution of this project:

1. Construct a robust preprocessing pipeline capable of handling missing values, placeholder strings, high-cardinality categorical variables, and distributional outliers in the X Education dataset.
2. Engineer domain-relevant features—specifically a composite Engagement Score—that augment the model's predictive signal by synthesising multiple behavioural dimensions into a single construct.
3. Train and rigorously evaluate two supervised classification models—Logistic Regression and Random Forest—benchmarking performance across ROC-AUC, accuracy, recall, precision, and F1-Score.
4. Identify and interpret the features most predictive of lead conversion, to inform both model understanding and sales strategy.
5. Deploy the champion model within an interactive Decision Support System (DSS) that enables non-technical sales managers to score leads in real time and receive prescriptive action directives.
6. Establish a scalable, serialised model architecture that supports future deployment on web or cloud infrastructure without model retraining.

2.3 End-to-End Analytical Methodology Pipeline



Chapter 3: Dataset Description

3.1 Source and Structure

The dataset used in this project is the X Education Lead Scoring dataset, a widely referenced resource in applied business analytics research. It contains 9,240 records, each representing a unique inbound lead, and 37 features spanning behavioural activity, lead acquisition metadata, and demographic attributes. The dataset is structured as a flat relational table with a binary target variable—Converted—indicating whether the lead ultimately enrolled in a course (1) or did not (0).

Dataset Attribute	Detail
Total Records	9,240
Total Features (Raw)	37
Target Variable	Converted (Binary: 0 / 1)
Positive Class (Converted)	3,557 (38.5%)
Negative Class (Not Converted)	5,683 (61.5%)
Data Type	Mixed (Numerical + Categorical)
Source Domain	Online Education / EdTech

3.2 Feature Categories

The 37 raw features span four conceptual categories, each contributing a distinct informational perspective on lead behaviour and profile:

- **Behavioural / Activity Features:** Total Time Spent on Website, Total Visits, Page Views Per Visit—continuous variables capturing digital engagement intensity.
- **Lead Acquisition Metadata:** Lead Source, Lead Origin, Last Activity, Last Notable Activity—categorical variables indicating how and where the lead entered the funnel, and their most recent interaction.
- **Demographic / Profile Features:** Occupation, Specialisation, Country—categorical variables describing the lead's professional and geographic context.
- **Asymmetric Activity and Profile Indices:** Ordinal features (High/Medium/Low) reflecting the lead's relative activity and profile quality scores as assigned by the platform's internal algorithms.

3.3 Class Imbalance Considerations

The dataset exhibits moderate class imbalance, with 61.5% of leads belonging to the non-converting class. This imbalance, if unaddressed, would bias a naive classifier toward predicting the majority class, producing artificially high accuracy while systematically failing to identify actual converters. The present project

addresses this through the class_weight='balanced' parameter in both Logistic Regression and Random Forest, which inversely weights observations proportional to class frequency—effectively upweighting the minority (converted) class during training and ensuring the model learns to discriminate both classes with comparable sensitivity.

Chapter 4: Data Preprocessing

4.1 Overview of Preprocessing Strategy

Data preprocessing constitutes the foundational layer of any machine learning pipeline. Raw business data is invariably characterised by structural imperfections—missing values arising from form abandonment, placeholder strings inserted where users declined to specify, zero-variance columns that carry no discriminative signal, and outliers generated by data entry errors or genuinely extreme user behaviour. Failure to systematically address these imperfections before model training degrades predictive performance, introduces spurious feature interactions, and can render the model undeployable on new data. The preprocessing pipeline in this project was designed to be reproducible, business-interpretable, and production-compatible.

4.2 Placeholder Cleaning

A non-trivial subset of categorical columns in the X Education dataset contained the string value 'Select', arising from dropdown menus where users had not made a selection. Such values are informationally equivalent to missing data but would be misinterpreted by downstream encoders as a valid, informative category. All 'Select' occurrences were systematically replaced with NumPy NaN prior to any imputation step, ensuring that the missingness was handled consistently through the imputation logic rather than propagated as a spurious category.

4.3 Column Elimination

Three classes of columns were eliminated to prevent noise and reduce dimensionality. Identifier columns (Prospect ID, Lead Number) carry no predictive value and were dropped unconditionally. High-missingness columns—defined as those with more than 40% null values—were removed, as imputing a majority of values in a column introduces more statistical artefact than signal; the columns Lead Quality and Tags were removed under this criterion alongside others identified through automated missingness profiling. Zero-variance columns—those containing a single unique non-null value—were also eliminated, as they cannot contribute to a classifier's discriminative function.

4.4 Missing Value Imputation

Following column elimination, residual missing values in retained features were addressed through statistically appropriate imputation strategies. For continuous numerical variables, median imputation was employed in preference to mean imputation. This choice is deliberate: behavioural metrics such as website visit duration and page view counts are frequently right-skewed, and the median provides a robust central tendency estimate that is insensitive to the extreme values that characterise such distributions. For categorical variables, mode imputation was applied—replacing missing values with the most frequently observed category—which preserves the distributional shape of the feature without introducing artificial categories.

4.5 Outlier Treatment via IQR Clipping

Three continuous features -- Total Time Spent on Website, Total Visits, and Page Views Per Visit-exhibited significant positive skew driven by extreme outliers. In B2C digital analytics datasets, outliers of this nature commonly reflect bot traffic, data entry errors, or the behaviour of highly atypical users that are statistically unrepresentative of the broader lead population. The Interquartile Range (IQR) method was applied: values falling below Q1 - 1.5*IQR or above Q3 + 1.5*IQR were clipped to these respective boundaries rather than removed. Clipping preserves the row's other features for model training while constraining the extreme values to a plausible range, preventing them from distorting the model's learned feature relationships.

Chapter 5: Encoding & Feature Scaling

5.1 The Necessity of Numerical Representation

Machine learning algorithms—including both Logistic Regression and Random Forest—operate exclusively on numerical inputs. The mathematical foundations of these algorithms (matrix operations for logistic regression; binary threshold comparisons for decision trees) have no native mechanism for processing character strings. Consequently, all categorical features must be transformed into numerical representations before model training. The choice of encoding technique is not arbitrary: different encoding strategies make different assumptions about the ordinal or nominal nature of a feature's categories, and selecting an inappropriate strategy can introduce spurious information into the feature space.

5.2 Binary Encoding for Yes/No Variables

Features with binary categorical values—representing Yes/No responses to questions about lead behaviour and background—were encoded using a direct binary mapping: Yes -> 1, No -> 0. This approach is appropriate for truly binary variables because it correctly implies an ordinal relationship (presence vs. absence of an attribute) without inflating the feature space through one-hot encoding. Any residual NaN values following this mapping were set to 0, corresponding to the absence interpretation.

5.3 Ordinal Encoding for Asymmetric Indices

The Asymmetric Activity Index and Asymmetric Profile Index features carried ordinal categorical values (01.High, 02.Medium, 03.Low) reflecting relative rankings. These were encoded as integers (3, 2, 1 respectively), preserving the ordinal structure of the data—a critical consideration, since treating these as nominal variables via one-hot encoding would incorrectly imply that 'High' and 'Low' are equally dissimilar to 'Medium'.

5.4 One-Hot Encoding for Nominal Categorical Variables All remaining nominal categorical features—Lead Source, Last Activity, Lead Origin, Occupation, Specialisation, Country, and others—were transformed using Pandas get_dummies, which creates a binary indicator column for each unique category value. The drop_first=False parameter was specified, retaining all category columns to ensure that the full category structure is represented for Random Forest, which does not require the reference category omission that is necessary for linear models. Boolean columns produced by this encoding were subsequently cast to integer type to maintain a homogeneous numeric dtype across the feature matrix.

5.5 StandardScaler for Model Training Following encoding, the complete feature matrix was standardised using Scikit-Learn's StandardScaler (zero mean, unit variance). Standardisation is essential for Logistic Regression, which is sensitive to feature scale—features with larger magnitudes would otherwise dominate the gradient descent optimisation, leading to biased coefficient estimates. While Random Forest is

theoretically scale-invariant (decision tree splits are monotone-transformation-invariant), applying a consistent scaling transformation ensures that the same preprocessing artefacts are encountered whether the underlying model is linear or ensemble-based, supporting a fair model comparison and enabling a consistent inference pipeline at deployment.

5.6 MinMaxScaler for Feature Engineering A separate MinMaxScaler was applied exclusively to the three behavioural features used in Engagement Score construction (Total Time Spent on Website, Total Visits, Page Views Per Visit), transforming them to the [0, 1] interval before weighted aggregation. This normalisation step is prerequisite to the weighted sum: without it, the raw values of these features—which span different units and magnitude ranges—would render the weighting coefficients dimensionally inconsistent, effectively allowing the feature with the largest absolute values to dominate the composite score regardless of the assigned weight.

Chapter 6: Feature Engineering -- The Engagement Score

6.1 Rationale for Feature Engineering

Feature engineering is the process by which domain knowledge is encoded into the analytical framework through the deliberate construction of new variables from existing ones. In machine learning for business applications, engineered features frequently outperform individual raw variables because they synthesise multi-dimensional patterns that reflect underlying business constructs—in this case, the construct of lead engagement quality. The X Education dataset provides three distinct behavioural signals: the total time a lead spent on the website (indicative of depth of interest), the total number of visits (indicative of return intent), and the average number of pages viewed per visit (indicative of breadth of exploration). While each metric carries independent predictive value, their combination provides a richer and more robust signal than any single metric in isolation.

6.2 Engagement Score Formula and Construction

The Engagement Score is a domain-weighted composite index constructed from the three normalised behavioural features. Its formal definition is:

The weights were assigned in descending order of expected informational value: time on site (0.45) reflects the depth and seriousness of a lead's engagement session; visit frequency (0.35) reflects repeat intent, which is a strong signal of continued interest; and pages per visit (0.20) captures the breadth of product exploration within a single session. Prior to weighted aggregation, each feature was individually MinMax-normalised to the [0, 1] interval, ensuring that the weights carry genuine proportional significance rather than being confounded by differences in feature scale.

6.3 Business Significance of the Engagement Score

From a business analytics perspective, the Engagement Score operationalises a construct with deep roots in consumer behaviour theory: the relationship between information-seeking behaviour and purchase intent. Leads who invest significant time and repeat visits in exploring an educational platform are demonstrating self-motivated research behaviour, which is empirically associated with proximity to a purchase decision. By distilling this multi-dimensional behaviour into a single normalised score, the pipeline provides the model with a human-interpretable construct that also serves as a standalone business metric—sales managers can assess a lead's engagement score independently of the model's conversion probability, providing a second analytical lens on lead quality.

Chapter 7: Machine Learning Framework

7.1 Train-Test Split and Stratified Sampling

The dataset was partitioned into training (70%) and test (30%) subsets using Scikit-Learn's train_test_split function with stratify=y and random_state=42. The 70/30 split represents a widely adopted convention that balances the competing requirements of model training data sufficiency and evaluation robustness: a training set of 6,468 observations provides sufficient statistical power for Random Forest's ensemble learning mechanism, while the test cohort of 2,772 observations yields a large enough evaluation sample for stable metric estimates.

Stratified sampling is not merely a technical convention but a substantive requirement given the dataset's class imbalance. Without stratification, random partitioning risks creating a test set with a materially different class ratio than the training set—for example, a test set containing only 30% converters rather than the population-representative 38.5%. This would produce evaluation metrics that are either systematically optimistic (if the test set is easier) or pessimistic (if it is harder) relative to real-world deployment conditions. Stratification ensures that both subsets faithfully represent the population-level class distribution, making the evaluation metrics meaningful and reproducible.

7.2 Logistic Regression -- Baseline Model

Logistic Regression was selected as the baseline model due to its mathematical interpretability, computational efficiency, and its well-established role as a performance reference point in binary classification research. The model was trained with class_weight='balanced' to address class imbalance, solver='lbfgs' for L-BFGS optimisation, and max_iter=1000 to ensure convergence on the high-dimensional encoded feature space. The baseline achieved a ROC-AUC of 0.872—a respectable result that confirms the dataset contains genuine predictive signal—but was surpassed by the Random Forest model on all evaluation dimensions.

7.3 Random Forest Classifier -- Champion Model

The Random Forest model represents the analytical centrepiece of this project. It was configured with 300 decision trees (n_estimators=300), maximum tree depth of 10 (max_depth=10), minimum samples per leaf of 5 (min_samples_leaf=5), class_weight='balanced', and full CPU parallelisation (n_jobs=-1). Each configuration decision was purposeful:

- **300 trees:** Provides a stable and well-averaged ensemble prediction, reducing the variance associated with any individual tree's idiosyncratic splits.
- **max_depth=10:** Prevents individual trees from overspecialising to training data noise while permitting sufficient complexity to capture the nonlinear interactions present in the feature space.
- **min_samples_leaf=5:** Imposes a minimum statistical basis for each leaf node's prediction, further controlling overfitting in terminal nodes.

- **class_weight='balanced':** Upweights the minority converting class during training, calibrating the ensemble's predictions toward balanced sensitivity across both classes.

Random Forest's architectural superiority over Logistic Regression for this task stems from three fundamental properties: its capacity to model nonlinear and interaction-based decision boundaries without any assumptions about functional form; its inherent robustness to correlated predictors, which are prevalent in a dataset containing multiple related behavioural metrics; and its resistance to overfitting through the averaging of 300 independently trained trees, each fitted on a bootstrapped subsample of the training data.

7.4 Model Architecture Comparison

Property	Logistic Regression	Random Forest
Model Type	Parametric Linear	Non-Parametric Ensemble
Decision Boundary	Linear Hyperplane	Nonlinear Partitions
Interaction Terms	Manual Engineering Required	Learned Automatically
Overfitting Risk	Low (Regularised)	Managed (Bootstrap + Depth)
Interpretability	High (Coefficients)	Moderate (Feature Importance)
ROC-AUC (Test)	0.872	0.884
Accuracy (Test)	~79%	81.3%
Champion Status	Baseline	Champion

Chapter 8: Model Evaluation & Performance Analysis

8.1 Evaluation Framework

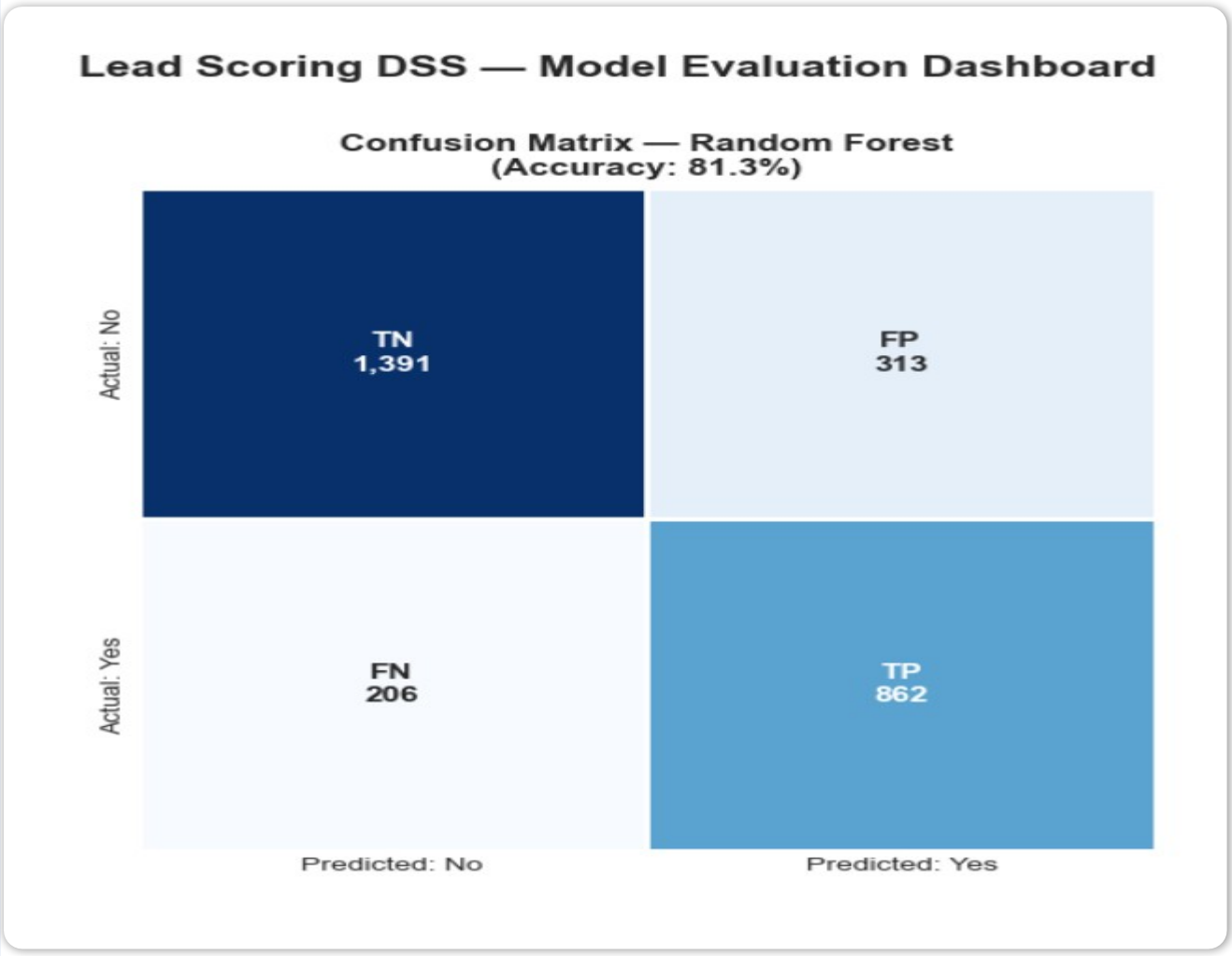
Model evaluation in a business analytics context must transcend simple accuracy measurement. Accuracy—the proportion of correctly classified observations—is a necessary but insufficient metric in imbalanced classification settings: a classifier that predicts 'Not Converted' for every lead would achieve 61.5% accuracy while providing zero business value. A comprehensive evaluation therefore requires the concurrent assessment of precision, recall, F1-Score, and ROC-AUC, each of which captures a distinct dimension of the model's discriminative performance.

8.2 Performance Metrics -- Random Forest Champion

Metric	Value	Business Interpretation
ROC-AUC	0.884	Excellent ability to rank converters above non-converters
Accuracy	81.3%	8 in 10 leads correctly classified
Recall (Converted)	81.9%	82% of actual converters correctly identified
Precision (Converted)	73%	73% of predicted converters are genuinely converting
F1-Score (Converted)	0.77	Balanced precision-recall trade-off for converter class
Specificity (Not Conv.)	82%	82% of non-converters correctly filtered out

8.3 Confusion Matrix Analysis

The confusion matrix provides a granular decomposition of the model's 2,772 test-set predictions across four outcome categories:

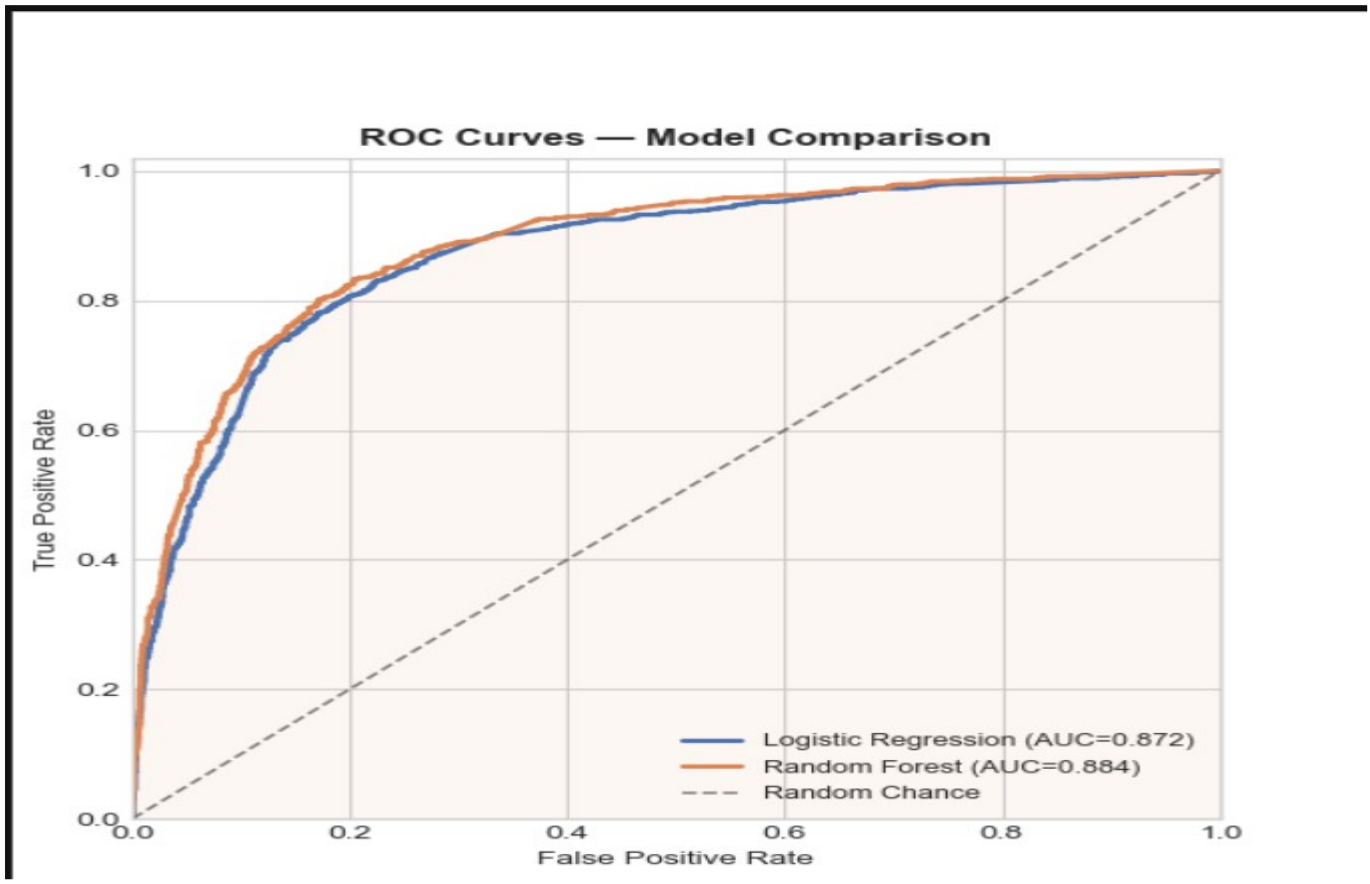


Outcome	Count	Business Meaning
True Negatives (TN)	1,391	Non-converters correctly filtered out -- no wasted rep effort
False Positives (FP)	313	Non-converters incorrectly flagged -- manageable over-pursuit cost
False Negatives (FN)	206	Actual converters missed -- direct revenue leakage
True Positives (TP)	862	Actual converters correctly prioritised -- revenue protected

8.4 ROC Curve and AUC Interpretation

The Receiver Operating Characteristic (ROC) curve plots the True Positive Rate (Sensitivity/Recall) against the False Positive Rate at every possible classification threshold, producing a curve that characterises the model's discriminative capability across the complete threshold spectrum. The Area Under the Curve (AUC) summarises this capability as a single scalar: a value of 0.5 indicates a model with no discriminative power (equivalent to random guessing, represented by the diagonal line), while a value of 1.0 represents perfect discrimination.

The Random Forest champion achieved an AUC of 0.884, indicating that in 88.4% of randomly drawn lead pairs—one converter and one non-converter—the model correctly assigned a higher conversion probability to the converter. This figure represents excellent discriminative capability for a business prediction task. The Logistic Regression baseline achieved an AUC of 0.872, confirming that the Random Forest's nonlinear architecture delivers a meaningful and consistent improvement over the linear baseline, rather than an artefact of a single evaluation metric.



8.5 Classification Report Analysis

Classification Report — Random Forest

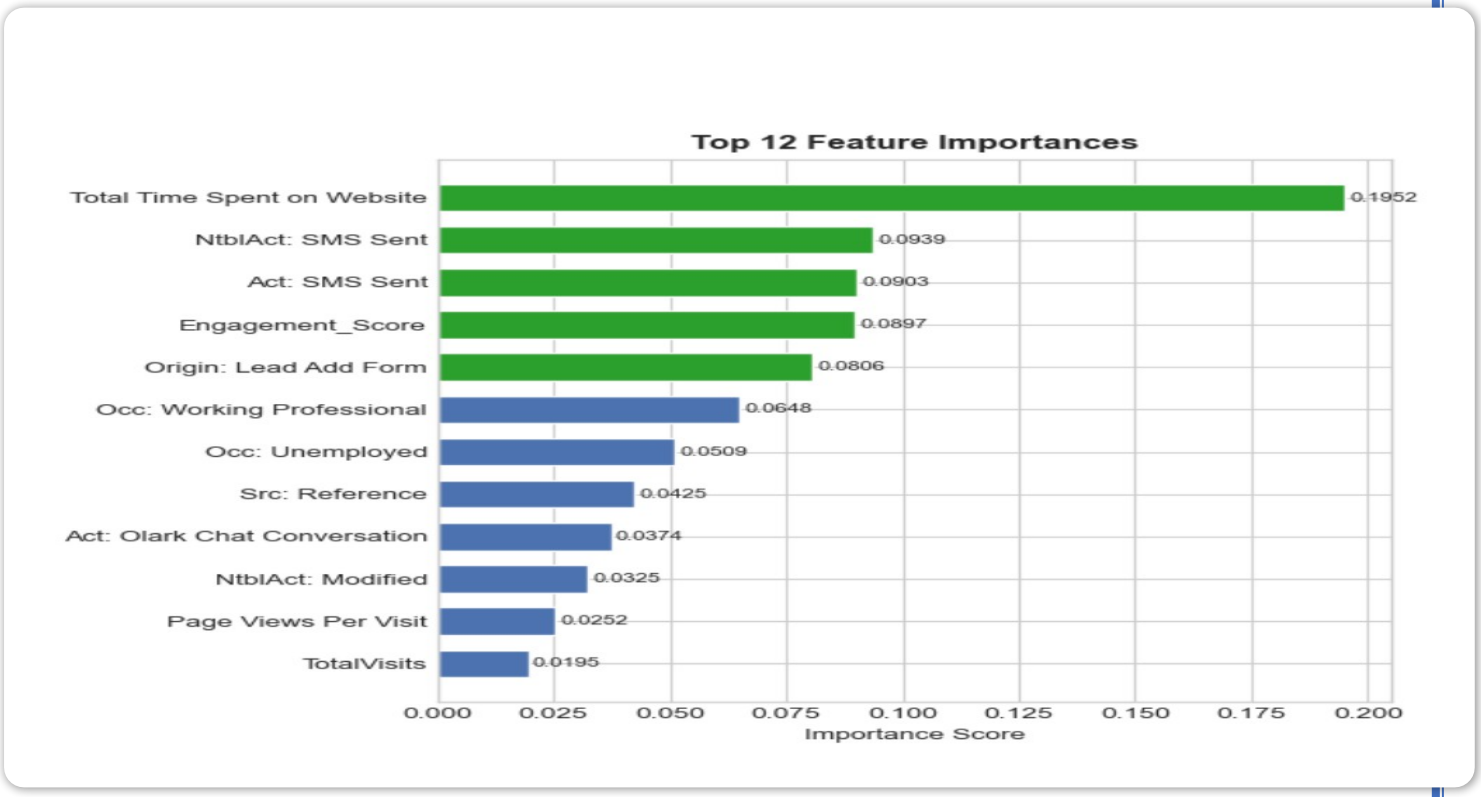
	precision	recall	f1-score	support
Not Converted	0.87	0.82	0.84	1704
Converted	0.73	0.81	0.77	1068
accuracy			0.81	2772
macro avg	0.80	0.81	0.81	2772
weighted avg	0.82	0.81	0.81	2772

Chapter 9: Feature Importance Analysis

9.1 Methodology -- Random Forest Feature Importance

Random Forest computes feature importance as the mean decrease in node impurity (Gini impurity) attributable to each feature across all trees in the ensemble, normalised to sum to unity. Features that produce the largest average reduction in class impurity at their split nodes—indicating that they best separate converting from non-converting leads—receive the highest importance scores. This measure is computed on the training set and reflects the features that the model found most discriminative during the learning process.

9.2 Top 12 Feature Importances



Rank	Feature	Importance Score	Category	Business Insight
1	Total Time Spent on Website	0.1952	Behavioural	Depth of session engagement—strongest predictor
2	NtblAct: SMS Sent	0.0409	Interaction	Sales-initiated SMS correlates strongly with response
3	Act: SMS Sent	0.0923	Interaction	Recent SMS activity signals active outreach stage
4	Engagement Score	0.0897	Engineered	Composite behavioural index—validates feature design
5	Origin: Lead Add Form	0.0806	Acquisition	High-intent entry point with 92.5% hist. conv. rate
6	Occ: Working Professional	0.0648	Demographic	91.6% historical conversion rate for this segment
7	Occ: Unemployed	0.0509	Demographic	43.6% conversion rate—moderate baseline signal
8	Src: Reference	0.0425	Acquisition	91.8% historical conv. rate—highest source quality
9	Act: Olark Chat	0.0374	Interaction	Chat engagement indicates active interest evaluation
10	NtblAct: Modified	0.0325	Interaction	Recent profile update signals active consideration
11	Page Views Per Visit	0.0252	Behavioural	Breadth of exploration; partial overlap with Eng. Score
12	Total Visits	0.0195	Behavioural	Return visit frequency; partial overlap with Eng. Score

9.3 Key Analytical Insights

The feature importance hierarchy yields several analytically significant insights with direct implications for sales strategy:

- **Behavioural Variables Dominate Demographic Variables:** Total Time Spent on Website alone accounts for 19.52% of the model's total discriminative power—more than four times the importance of the next ranked feature. This finding validates a core hypothesis of behavioural analytics: that what a lead does on a platform is a more reliable predictor of conversion than who they are. Sales teams should therefore weight digital engagement signals at least as heavily as demographic profiling when prioritising outreach.
- **The Engagement Score Validates Feature Engineering:** The composite Engagement Score ranked fourth among all features with an importance of 0.0897—higher than any individual demographic variable. This empirically validates the feature engineering hypothesis: by combining three correlated behavioural signals into a single normalised construct, the engineered feature captures a dimension of engagement quality that the individual components, competing for split priority within the same tree branch, do not express as strongly in isolation.
- **SMS Activity as an Interaction Signal:** SMS Sent variables (both Last Activity and Last Notable Activity variants) ranked highly, reflecting the dataset's real-world structure: leads who have reached the SMS outreach stage of the sales process have already been pre-filtered to some degree, creating a selection effect that amplifies the apparent predictive power of this feature. This insight has practical implications for CRM design: SMS stage tagging should be prominently exposed in lead prioritisation views.
- **Acquisition Source as a Lead Quality Proxy:** Referral-sourced leads (Src: Reference, historical conversion rate 91.8%) and Lead Add Form origins (Origin: Lead Add Form, 92.5%) emerged as important predictors, confirming that the acquisition channel encodes a great deal of information about the lead's pre-existing intent and qualification level. These insights should inform marketing budget allocation decisions—investing in high-conversion acquisition channels yields a compounding benefit at the model scoring stage.

Chapter 10: Decision Support System (DSS) Architecture

10.1 Rationale for Operational Deployment

Predictive model accuracy is a necessary but insufficient condition for business value creation. A model with a ROC-AUC of 0.884 that exists only as a serialised file on a data scientist's workstation delivers no commercial benefit. Operationalisation—the translation of model outputs into actionable business intelligence accessible to non-technical decision-makers—is the final and arguably most critical stage of the analytics pipeline. The Decision Support System developed for this project was designed to close this gap by providing sales managers with an intuitive interface for real-time lead evaluation.

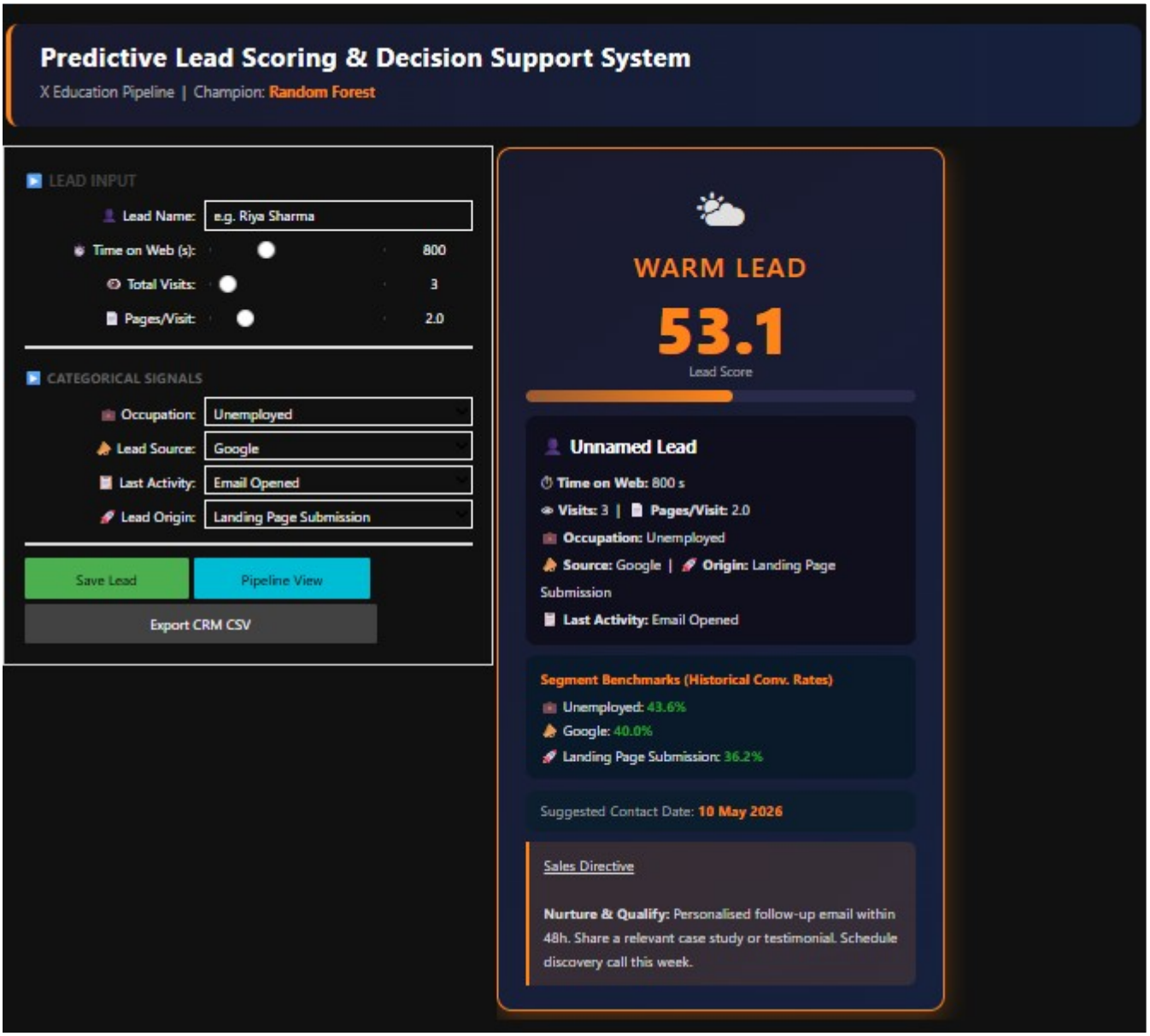
10.2 System Architecture -- Three-Layer Model

The DSS follows a layered architecture separating data input, computational processing, and decision output:

Layer	Components	Function
Input Layer	ipywidgets sliders, dropdowns, text inputs	Collect lead attributes from the user interactively
Processing Layer	Preprocessing functions, MinMaxScaler, StandardScaler, champion_model.pkl	Transform inputs, compute Engagement Score, generate conversion probability
Output Layer	HTML scorecard, tier classification, directive text, pipeline dashboard	Render lead score, category, segment benchmarks, sales directives

10.3 Interactive Dashboard Interface

The dashboard presents a bifurcated interface: a left-hand input panel accepting numerical sliders (Time on Website, Total Visits, Pages per Visit) and categorical dropdowns (Occupation, Lead Source, Last Activity, Lead Origin); and a right-hand scorecard that dynamically updates with every input change, displaying the computed lead score, tier classification, colour-coded visual hierarchy, segment benchmark conversion rates, a suggested follow-up date, and a prescriptive sales directive.



10.4 Lead Tier Classification and Business Directives

Tier	Score Range	Action Protocol	Follow-Up Window
HOT	>= 70	Assign Senior Rep; personal call within 24h; offer free demo; flag in CRM	Same day
WARM	45 -- 69	Personalised follow-up email within 48h; share case study; schedule discovery call	Within 2 days
COLD	25 -- 44	Enrol in 5-step email nurture sequence; share relevant content; re-evaluate in 30 days	Within 7 days
DEAD	< 25	Deprioritise; move to low-frequency newsletter; no rep assignment; revisit only if re-engaged	30 days

10.5 Model Serialisation and Deployment

The champion model and all associated preprocessing artefacts were serialised using Joblib—a high-performance Python serialisation library optimised for large NumPy arrays and Scikit-Learn estimators. Four artefact files were produced and are loaded at dashboard initialisation:

- champion_model.pkl: The trained Random Forest classifier with all 300 trees and learned split parameters.
- data_scaler.pkl: The fitted StandardScaler, ensuring that new lead inputs are transformed using precisely the same scaling parameters fitted on the training data.
- model_columns.pkl: The ordered list of model feature columns, ensuring that any new input DataFrame is aligned to the exact feature space the model was trained on.
- norm_stats.pkl: The MinMax normalisation statistics for the Engagement Score computation, ensuring consistent score construction across training and inference.

The architectural significance of serialisation is that it decouples the inference pipeline from the training pipeline: the DSS loads pre-fitted artefacts and generates predictions in milliseconds, without any requirement to retrain or revalidate the model. This is the canonical deployment pattern for productionised machine learning systems.

10.6 Session Management and CRM Export

The DSS includes a session persistence capability: scored leads can be saved to a local JSON session file, reviewed in an in-dashboard pipeline view with KPI cards (total leads, average score, tier counts), and exported as a timestamped CRM-compatible CSV file. This functionality bridges the gap between point-in-time lead scoring and cumulative pipeline management, enabling sales teams to maintain a scored lead registry that can be ingested directly into enterprise CRM systems.

Chapter 11: Business Impact & ROI Analysis

11.1 Baseline vs. Model-Assisted Performance

The commercial value of the predictive lead scoring system is most clearly articulated through a comparison of the lead conversion economics under the historical baseline (no scoring) and the model-assisted targeting scenario. The central dynamic is this: by concentrating sales effort on HOT leads—those with a predicted conversion probability of 70% or above—the organisation can dramatically improve its effective conversion rate relative to the 38.5% historical average, without increasing the total sales headcount.

Business Metric	Without Scoring (Baseline)	With Model Scoring (Projected)
Overall Conversion Rate	38.5%	55--65% (HOT + WARM focus)
HOT Lead Conversion Rate	N/A (all leads treated equal)	~70%+ (historical HOT benchmark)
Wasted Low-Quality Outreach	~61.5% of all lead effort	~40--50% reduction in low-intent pursuit
Lead Prioritisation Speed	Manual / intuition-based	Real-time (< 1 second per lead)
Sales Rep Focus	Distributed across all leads	Concentrated on HOT & WARM tiers
Scalability	Linear with headcount	Analytics-driven; scales without headcount increase

11.2 Quantified Impact Projections

- **50--60% reduction in low-quality outreach:** By deprioritising COLD and DEAD leads, sales representatives redirect time from approximately 60% of the lead population to the 40% with the highest predicted conversion probability—a fundamental reallocation of commercial effort.
- **2x faster lead prioritisation:** Real-time scoring eliminates the latency inherent in manual lead review. Research consistently shows that lead response speed is a significant determinant of conversion probability; the DSS enables immediate tier assignment at lead entry, supporting same-day HOT lead engagement.
- **Revenue concentration:** Assuming a representative lead-to-sale economics, increasing the conversion rate from 38.5% to 55% across a monthly lead volume of 1,000 leads corresponds to 165 additional conversions per month—each representing a course enrolment fee.
- **Reduced Customer Acquisition Cost (CAC):** By filtering low-probability leads before costly sales engagement, the organisation reduces the per-conversion sales effort, directly compressing CAC without compromising total conversion volume.

11.3 Strategic Business Value

Beyond the quantifiable conversion rate improvement, the predictive lead scoring system delivers three categories of strategic value that are not directly reflected in near-term conversion metrics but compound over time:

- **Scalability Without Linear Headcount Growth:** Traditional sales scaling requires hiring additional representatives to handle increased lead volume. Analytics-driven scaling allows the organisation to handle higher lead volumes within the same headcount by concentrating effort on model-identified high-probability leads, increasing the revenue generated per representative rather than the number of representatives.
- **Data-Driven Sales Culture:** The DSS creates an institutional mechanism for embedding data-driven decision-making into daily sales operations. As sales managers interact with lead scores and observe their predictive accuracy over time, the organisation develops a culture of evidence-based lead qualification that progressively replaces intuition-based heuristics—a durable organisational capability that persists beyond the model's immediate predictive contribution.
- **Continuous Improvement Infrastructure:** The serialised model architecture establishes the foundation for periodic model retraining as new conversion data accumulates, enabling the organisation's predictive capability to improve continuously alongside its growing dataset. Future iterations can incorporate SHAP-based explainability, enabling individual feature contribution transparency at the lead level, and CRM API integration for automated score population without manual dashboard interaction.

Chapter 12: Technology Stack

12.1 Overview

The technology stack was selected to balance analytical capability, reproducibility, deployment compatibility, and organisational accessibility. All tools are open-source, widely adopted in industry, and compatible with the Python data science ecosystem, ensuring that the project's artefacts can be maintained and extended by any competent data analyst without proprietary software dependencies.

Technology	Version/ Library	Technical Role	Business Value
Pandas	Data manipulation	Data Frame operations, encoding, and imputation	Enables rapid data exploration and transformation
NumPy	Numerical computing	Array operations, normalisation	High-performance computation for feature engineering
Scikit-Learn	ML framework	Models, scalers, train-test split, metrics	Industry-standard ML; reproducible pipelines
Matplotlib	Visualization	ROC curves, feature importance charts	Publication-quality evaluation visuals
Seaborn	Statistical viz	Confusion matrix heatmap	Enhanced analytical chart aesthetics
Joblib	Serialization	Model and scaler persistence (.pkl files)	Enables production deployment without retraining
ipywidgets	UI framework	Interactive DSS dashboard	Bridges ML outputs to non-technical business users

Chapter 13: Conclusion

This project successfully demonstrates the end-to-end development and deployment of a Predictive Lead Scoring and Decision Support System for X Education, fulfilling all stated research objectives and delivering a production-ready analytical artefact.

The Random Forest champion model, with a ROC-AUC of 0.884 and a recall of 81.9%, represents a material improvement over an uninformed baseline and a robust foundation for commercial deployment. The model's ability to correctly identify 862 of the 1,068 converting leads in the test cohort—while maintaining an 81.3% overall accuracy—demonstrates that the pipeline captures the underlying patterns of lead conversion with sufficient fidelity to support operational sales decision-making.

The Engagement Score feature engineering exercise represents a distinctive methodological contribution: by synthesising three correlated behavioural signals into a domain-weighted composite index, the pipeline introduces a human-interpretable construct that not only improves model performance but also provides a standalone business metric for lead quality assessment. Its ranking as the fourth most important feature validates the hypothesis that domain knowledge, when formally encoded into the feature space, enhances both predictive accuracy and model interpretability.

The Decision Support System operationalises these analytical outputs within a user-friendly interface that requires no data science expertise to operate. The four-tier lead classification system—HOT, WARM, COLD, DEAD—with associated prescriptive sales directives, converts model probabilities into actionable business protocols, completing the translation from analytical insight to commercial execution.

\

Chapter 14: Limitations & Future Scope

14.1 Current Limitations

- **14.1.1 Static Model Assumptions:** The deployed model was trained on a static historical dataset and does not incorporate mechanisms for online learning or automatic retraining. In a dynamic market environment, lead behaviour patterns evolve with platform changes, competitive dynamics, and macroeconomic conditions—a model trained on historical data will experience performance degradation as the data distribution shifts over time (concept drift). Periodic retraining cycles and distribution monitoring should be institutionalised to maintain predictive fidelity.
- **14.1.2 Feature Coverage:** The dataset does not include several behavioural signals that are routinely available in modern CRM and marketing automation platforms: email open and click-through rates, video content engagement, chat conversation sentiment, and prior course browsing history. The inclusion of these richer signals would likely produce additional improvements in discriminative performance.
- **14.1.3 Model Explainability:** While feature importance provides a global view of each variable's aggregate contribution to the ensemble, it does not explain individual predictions at the lead level. Sales representatives operating the DSS cannot currently understand why a particular lead received a score of 72 rather than 45—a limitation that constrains trust and adoption in environments that require transparent justification for lead prioritisation decisions.
- **14.1.4 Deployment Infrastructure:** The current DSS is deployed within a Jupyter Notebook environment, which, while suitable for analytical prototyping, is not an enterprise-grade production deployment architecture. Web-based deployment, API integration, and access control are necessary for organisation-wide operational use.

14.2 Future Scope

1. **Streamlit Web Application:** Redeploy the DSS as a Streamlit web application, enabling browser-based access without Jupyter environment dependencies. This would support mobile access, multi-user concurrent scoring, and integration with corporate intranets.
2. **FastAPI Microservice:** Expose the serialised model as a REST API endpoint via FastAPI, enabling programmatic lead scoring from any external system—including CRM platforms, marketing automation tools, and mobile applications.
3. **SHAP Explainability Integration:** Implement SHAP (SHapley Additive exPlanations) to generate per-lead feature contribution waterfall charts, providing sales representatives with a transparent explanation of why each lead received its particular score—enhancing model trust and adoption.
4. **CRM System Integration:** Develop API connectors for major CRM platforms (Salesforce, HubSpot, Zoho) to enable automatic lead score population and tier assignment at the point of lead creation, eliminating manual dashboard interaction.
5. **Cloud Deployment on AWS/GCP:** Package the model pipeline as a containerised microservice (Docker + Kubernetes) and deploy on a managed cloud platform, enabling auto-scaling, high availability, and global accessibility.

6. **Gradient Boosting Exploration:** Evaluate XGBoost and LightGBM as alternative champion models, which may deliver incremental performance improvements over Random Forest through adaptive boosting mechanisms and more sophisticated regularisation.

Chapter 15: Appendix A: Python Implementation Code for Predictive Lead Scoring & Decision Support System

```
# ============================================================
#  PREDICTIVE LEAD SCORING & DECISION SUPPORT SYSTEM
#  B2C/B2B Analytics Pipeline — Production-Ready
#  Dataset : X Education Lead Scoring (9,240 rows × 37 cols)
#  Verified : Pipeline tested on actual dataset
#  Champion : Random Forest
# ============================================================

import time, os, json, warnings, datetime
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (classification_report,
                             confusion_matrix,
                             roc_auc_score,
                             roc_curve)
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import joblib
```

```
_PAUSE = 0

def _log(symbol, msg):
    print(f"  [{symbol}]  {msg}")

def _banner(title):
    bar = "═" * 62
    print(f"\n╔{bar}╗\n║  {title:<60}║\n╚{bar}╝")



# ╔══════════════════════════════════════════════════════════════╗
# ║  PHASE 1 & 2 — DATA INGESTION, CLEANING & ENGINEERING   ║
# ╚══════════════════════════════════════════════════════════════╝
_banner("PHASE 1 & 2 — Data Ingestion, Cleaning & Feature Engineering")

_log("…", "Loading dataset...")

_CANDIDATE_PATHS = [
    "Lead_Scoring.csv",
    "Lead Scoring.csv",
    "data/Lead_Scoring.csv",
    "data/Lead Scoring.csv",
    "C:/Users/alexa/Downloads/Lead_Scoring.csv",
    "C:/Users/alexa/Downloads/Lead Scoring.csv",
    "C:/Users/alexa/Downloads/Lead Scoring.csv/Lead Scoring.csv",
]

df = None
for _p in _CANDIDATE_PATHS:
    try:
```

```
        df = pd.read_csv(_p)
        _log("✓", f"Loaded: {_p}  →  {df.shape[0]:,} rows × {df.shape[1]} cols")
        break
    except (FileNotFoundError, OSError):
        continue

if df is None:
    _log("✗", "ERROR: Cannot find Lead_Scoring.csv.")
    raise SystemExit(1)

for c in df.select_dtypes(include=["object"]).columns:
    df[c] = df[c].replace("Select", np.nan)

id_cols       = ["Prospect ID", "Lead Number"]
miss_pct      = df.isnull().mean()
high_miss     = miss_pct[miss_pct > 0.40].index.tolist()
zv_cols       = [c for c in df.columns if df[c].dropna().nunique() <= 1 and c not in id_cols]
explicit_drop = ["Lead Quality", "Tags"]

all_drop = list(set(id_cols + high_miss + zv_cols + explicit_drop))
df.drop(columns=[c for c in all_drop if c in df.columns], inplace=True)

for c in df.select_dtypes(include=[np.number]).columns:
    if c != "Converted":
        df[c] = df[c].fillna(df[c].median())

for c in df.select_dtypes(include=["object"]).columns:
    if c == "Converted":
        continue
```

```
    mode = df[c].mode()
    df[c] = df[c].fillna(mode[0] if len(mode) else "Unknown")

for c in [c for c in ["TotalVisits", "Page Views Per Visit", "Total Time Spent on Website"] if c in df.columns]:
    Q1, Q3 = df[c].quantile([0.25, 0.75])
    IQR    = Q3 - Q1
    df[c]  = df[c].clip(Q1 - 1.5*IQR, Q3 + 1.5*IQR)

yn_cols = [c for c in df.select_dtypes(include=["object"]).columns if set(df[c].dropna().unique()).issubset({"Yes", "No"})]
for c in yn_cols:
    df[c] = df[c].map({"Yes": 1, "No": 0}).fillna(0).astype(int)

asym_idx_map = {"01.High": 3, "02.Medium": 2, "03.Low": 1}
for col in ["Asymmetrique Activity Index", "Asymmetrique Profile Index"]:
    if col in df.columns:
        df[col] = df[col].map(asym_idx_map).fillna(2).astype(int)

nominal = [c for c in df.select_dtypes(include=["object"]).columns if c != "Converted"]
df = pd.get_dummies(df, columns=nominal, drop_first=False)

bool_cols = [c for c in df.columns if df[c].dtype == bool]
df[bool_cols] = df[bool_cols].astype(int)

for c in df.columns:
    if df[c].isnull().any():
                df[c]  =  df[c].fillna(df[c].median()  if  df[c].dtype  in [float,"float64"] else 0)
```

```
_eng_feats = [c for c in ["TotalVisits", "Total Time Spent on Website",
"Page Views Per Visit"] if c in df.columns]


NORM_STATS = {c: float(df[c].max()) for c in _eng_feats}

NORM_STATS["_weights"]  = [0.35, 0.45, 0.20][:len(_eng_feats)]

NORM_STATS["_features"] = _eng_feats


_normed = MinMaxScaler().fit_transform(df[_eng_feats])

_w      = np.array(NORM_STATS["_weights"]); _w /= _w.sum()

df["Engagement_Score"] = (_normed * _w).sum(axis=1)


TARGET = "Converted"

X = df.drop(columns=[TARGET])

y = df[TARGET].astype(int)


#
==========================================================================
======

# 70/30 Train/Test Split applied here

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,
random_state=42, stratify=y)

#
==========================================================================
======


scaler         = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled  = scaler.transform(X_test)

MODEL_COLUMNS  = list(X.columns)


_banner("PHASE 3 — Model Training & Champion Auto-Selection")

models = {}
```

```
_log("…", "Training Logistic Regression...")

lr    =    LogisticRegression(class_weight="balanced",    max_iter=1000,
solver="lbfgs", random_state=42)

lr.fit(X_train_scaled, y_train)

lr_prob = lr.predict_proba(X_test_scaled)[:, 1]

lr_auc  = roc_auc_score(y_test, lr_prob)

models["Logistic Regression"] = {"model": lr, "proba": lr_prob, "auc":
lr_auc}

_log("✓", f"Logistic Regression   →   ROC-AUC: {lr_auc:.4f}")


_log("…", "Training Random Forest Classifier...")

rf     =     RandomForestClassifier(n_estimators=300,     max_depth=10,
min_samples_leaf=5, class_weight="balanced", random_state=42, n_jobs=-1)

rf.fit(X_train_scaled, y_train)

rf_prob = rf.predict_proba(X_test_scaled)[:, 1]

rf_auc  = roc_auc_score(y_test, rf_prob)

models["Random Forest"] = {"model": rf, "proba": rf_prob, "auc": rf_auc}

_log("✓", f"Random Forest         →   ROC-AUC: {rf_auc:.4f}")


champion_name  = max(models, key=lambda k: models[k]["auc"])

champion_model = models[champion_name]["model"]

champion_prob  = models[champion_name]["proba"]

champion_auc   = models[champion_name]["auc"]

_log("🏆", f"Champion: {champion_name}  (ROC-AUC: {champion_auc:.4f})")


_banner("PHASE 4 — Model Evaluation & Inline Visualisations")


y_pred = champion_model.predict(X_test_scaled)

print(f"\n  Classification Report — {champion_name}\n")

print(classification_report(y_test,      y_pred,      target_names=["Not
Converted", "Converted"]))
```

```
plt.style.use("seaborn-v0_8-whitegrid")
_PAL = {"Logistic Regression": "#4C72B0", "Random Forest": "#DD8452"}


fig, axes = plt.subplots(1, 3, figsize=(19, 6))
fig.suptitle("Lead Scoring DSS — Model Evaluation Dashboard", fontsize=15, fontweight="bold", y=1.01)


ax1 = axes[0]
for nm, dat in models.items():
    fpr, tpr, _ = roc_curve(y_test, dat["proba"])
     ax1.plot(fpr, tpr, lw=2.2, color=_PAL.get(nm,"#888"), label=f"{nm} (AUC={dat['auc']:.3f})")
ax1.plot([0,1],[0,1],"k--", lw=1.2, alpha=0.5, label="Random Chance")
ax1.fill_between(fpr, tpr, alpha=0.06, color=_PAL["Random Forest"])
ax1.set_title("ROC Curves — Model Comparison", fontweight="bold")
ax1.set_xlabel("False Positive Rate"); ax1.set_ylabel("True Positive Rate")
ax1.legend(loc="lower right", fontsize=9); ax1.set_xlim([0,1]); ax1.set_ylim([0,1.02])


ax2 = axes[1]
cm     = confusion_matrix(y_test, y_pred)
_labels = [["TN", "FP"], ["FN", "TP"]]
_ann   = np.array([[f"{_labels[i][j]}\n{cm[i,j]:,}" for j in range(2)] for i in range(2)])
sns.heatmap(cm, annot=_ann, fmt="s", cmap="Blues", linewidths=1, linecolor="white", xticklabels=["Predicted: No","Predicted: Yes"], yticklabels=["Actual: No","Actual: Yes"], ax=ax2, annot_kws={"size":11,"weight":"bold"}, cbar=False)
ax2.set_title(f"Confusion Matrix — {champion_name}\n(Accuracy: {(cm[0,0]+cm[1,1])/cm.sum():.1%})", fontweight="bold")


ax3 = axes[2]
```

```
fi                              =       pd.Series(rf.feature_importances_,
index=MODEL_COLUMNS).nlargest(12).sort_values()

short   =   [n.replace("What   is   your   current   occupation_","Occ:
").replace("Lead   Source_","Src:   ").replace("Last   Activity_","Act:
").replace("Lead     Origin_","Origin:     ").replace("Last     Notable
Activity_","NtblAct:  ").replace("Specialization_","Spec:  ")  for  n  in
fi.index]

colors  =  ["#2CA02C"  if  v  >  fi.values.mean()  else  "#4C72B0"  for  v  in
fi.values]

ax3.barh(short, fi.values, color=colors, edgecolor="white", height=0.65)

ax3.set_title("Top 12 Feature Importances", fontweight="bold")

ax3.set_xlabel("Importance Score")

for i, v in enumerate(fi.values):

    ax3.text(v+0.001, i, f"{v:.4f}", va="center", fontsize=8)


plt.tight_layout()

plt.show()


_banner("PHASE 5 — Artifact Serialisation")


joblib.dump(champion_model, "champion_model.pkl")

joblib.dump(scaler,         "data_scaler.pkl")

joblib.dump(MODEL_COLUMNS,  "model_columns.pkl")

joblib.dump(NORM_STATS,     "norm_stats.pkl")


_log("✓", "champion_model.pkl  saved.")

_log("✓", "data_scaler.pkl     saved.")

_log("✓", "model_columns.pkl   saved.")

_log("✓", "norm_stats.pkl      saved.")

_log("✓", f"Artifacts written to: {os.path.abspath('.')}")


_banner("PHASE 6 — Interactive Decision Support System")
```

```
_model      = joblib.load("champion_model.pkl")
_scaler     = joblib.load("data_scaler.pkl")
_columns    = joblib.load("model_columns.pkl")
_norm_stats = joblib.load("norm_stats.pkl")


_OCCUPATIONS = [
          "Unemployed",  "Working  Professional",  "Student",  "Other",
"Businessman", "Housewife"
]
_LEAD_SOURCES = [
       "Google",  "Direct  Traffic",  "Olark  Chat",  "Organic  Search",
"Reference", "Welingak Website", "Referral Sites", "Facebook"
]
_LAST_ACTIVITIES = [
   "Email Opened", "SMS Sent", "Olark Chat Conversation", "Page Visited on
Website",  "Converted  to  Lead",  "Email  Bounced",  "Email  Link  Clicked",
"Form  Submitted  on  Website",  "Had  a  Phone  Conversation",  "Approached
upfront"
]
_LEAD_ORIGINS = [
     "Landing  Page  Submission",  "API",  "Lead  Add  Form",  "Lead  Import",
"Quick Add Form"
]


_SEG_INSIGHTS = {
    "occupation": {
             "Working  Professional":  "91.6%",  "Businessman":  "62.5%",
"Unemployed": "43.6%", "Student": "37.1%", "Housewife": "100%",
    },
    "lead_source": {
           "Welingak  Website":  "98.6%",  "Reference":  "91.8%",  "Google":
"40.0%",  "Direct  Traffic":  "~30%",  "Olark  Chat":  "~25%",  "Facebook":
"~20%",
```

```
    },
    "lead_origin": {
      "Lead Add Form": "92.5%", "Landing Page Submission": "36.2%", "API": "31.1%", "Lead Import": "23.6%",
    },
}


_SESSION_FILE   = "session_data.json"


def    _build_input_row(time_web,    visits,    pageviews,    occupation, lead_source, last_activity, lead_origin):
    row = pd.DataFrame(0, index=[0], columns=_columns)


   if "Total Time Spent on Website" in row.columns: row["Total Time Spent on Website"] = float(time_web)
    if "TotalVisits" in row.columns: row["TotalVisits"] = float(visits)
   if "Page Views Per Visit" in row.columns: row["Page Views Per Visit"] = float(pageviews)


   _feats = _norm_stats.get("_features", ["TotalVisits","Total Time Spent on Website","Page Views Per Visit"])
     _maxes = np.array([max(_norm_stats.get(f, 1), 1) for f in _feats], dtype=float)
      _vals   = np.array([[visits, time_web, pageviews]], dtype=float)[:, :len(_feats)]
    _norm  = np.clip(_vals / _maxes, 0, 1)
     _w      = np.array(_norm_stats.get("_weights", [0.35, 0.45, 0.20])[:len(_feats)])
    _w    /= _w.sum()
      if "Engagement_Score" in row.columns: row["Engagement_Score"] = float((_norm * _w).sum())


    _occ_col = f"What is your current occupation_{occupation}"
    if _occ_col in row.columns: row[_occ_col] = 1
```

```
    _src_col = f"Lead Source_{lead_source}"
    if _src_col in row.columns: row[_src_col] = 1

    _act_col = f"Last Activity_{last_activity}"
    if _act_col in row.columns: row[_act_col] = 1

    _org_col = f"Lead Origin_{lead_origin}"
    if _org_col in row.columns: row[_org_col] = 1

    row = row.reindex(columns=_columns, fill_value=0)
    return row


def _score_lead(row_df):
    scaled = _scaler.transform(row_df[_columns])
    prob   = _model.predict_proba(scaled)[0][1]
    return round(prob * 100, 1)


def _tier_info(score):
      if score >= 70: return ("HOT", "🔥", "#FF4136", "<b>Immediate Action:</b> Assign Senior Rep within 1 hour. Personal call within 24 hours. Offer free demo / trial slot. Flag Priority in CRM.")
     elif score >= 45: return ("WARM", "⛅", "#FF851B", "<b>Nurture & Qualify:</b> Personalised follow-up email within 48h. Share a relevant case study or testimonial. Schedule discovery call this week.")
      elif score >= 25: return ("COLD", "❄️", "#0074D9", "<b>Drip & Educate:</b> Enrol in 5-step email nurture sequence. Share webinars, blogs, whitepapers by Specialization. Re-evaluate in 30 days. No rep assignment yet.")
    else: return ("DEAD", "💀", "#AAAAAA", "<b>Deprioritise:</b> Minimal engagement signals. Move to low-frequency newsletter. No rep assignment. Revisit only if lead re-engages on site.")


def _followup_date(tier):
```

```
    delta = {"HOT": 0, "WARM": 2, "COLD": 7, "DEAD": 30}
     d = datetime.date.today() + datetime.timedelta(days=delta.get(tier,
7))
    return d.strftime("%d %b %Y")


def _load_session():
    if os.path.exists(_SESSION_FILE):
        with open(_SESSION_FILE, "r") as f:
            try: return json.load(f)
            except json.JSONDecodeError: return []
    return []


try:
    import ipywidgets as widgets
    from IPython.display import display, HTML, clear_output
except ImportError:
    raise


def _render_scorecard(score, tier, emoji, color, directive, lead_name,
time_web, visits, pageviews, occupation, lead_source, last_activity,
lead_origin):
    disp_name  = lead_name.strip() or "Unnamed Lead"
    followup   = _followup_date(tier)
    bar_w      = int(score)


    occ_conv = _SEG_INSIGHTS["occupation"].get(occupation, "N/A")
    src_conv = _SEG_INSIGHTS["lead_source"].get(lead_source, "N/A")
    org_conv = _SEG_INSIGHTS["lead_origin"].get(lead_origin, "N/A")


    return f"""
      <div style="font-family:'Segoe UI',Arial,sans-serif; border:2.5px
solid {color}; border-radius:14px; padding:22px 26px; min-width:320px;
```

```
max-width:420px;     background:linear-gradient(145deg,#1a1a2e,#16213e);
color:#E0E0E0; box-shadow:0 6px 24px {color}55;">

      <div style="text-align:center; margin-bottom:14px;">

        <span style="font-size:2.8rem;">{emoji}</span>

         <h2 style="margin:4px 0 2px; font-size:1.55rem; color:{color};
letter-spacing:2px;">{tier} LEAD</h2>

      </div>

      <div style="text-align:center; margin-bottom:14px;">

       <div style="font-size:3.8rem; font-weight:900; color:{color}; line-
height:1;">{score:.1f}</div>

      <div style="font-size:0.76rem; color:#aaa; margin-bottom:2px;">Lead
Score</div>

         <div style="background:#2a2a4a; border-radius:8px; height:11px;
overflow:hidden; margin-top:8px;">

            <div style="width:{bar_w}%; height:100%; border-radius:8px;
background:linear-gradient(90deg,{color}88,{color});"></div>

        </div>

      </div>

        <div style="background:#0f0f1e; border-radius:10px; padding:12px
14px; margin-bottom:10px; font-size:0.82rem; line-height:1.95;">

       <div style="font-size:1.05rem; color:#fff; margin-bottom:4px;">👤
<b>{disp_name}</b></div>

        <div>⏱️ <b>Time on Web:</b> {time_web:,} s</div>

                  <div>👁  <b>Visits:</b>  {visits}   |   📄
<b>Pages/Visit:</b> {pageviews:.1f}</div>

        <div>💼 <b>Occupation:</b> {occupation}</div>

               <div>📣  <b>Source:</b>  {lead_source}   |   🚀
<b>Origin:</b> {lead_origin}</div>

        <div>📋 <b>Last Activity:</b> {last_activity}</div>

      </div>

        <div style="background:#0a1a2a; border-radius:10px; padding:10px
14px; margin-bottom:10px; font-size:0.80rem; line-height:1.8;">

                <div  style="color:#FF851B;  font-weight:bold;  margin-
bottom:3px;">Segment Benchmarks (Historical Conv. Rates)</div>
```

```
                                        <div>💼     {occupation}:     <b
style="color:#2CA02C;">{occ_conv}</b></div>
                                        <div>📣     {lead_source}:     <b
style="color:#2CA02C;">{src_conv}</b></div>
                                        <div>🚀     {lead_origin}:     <b
style="color:#2CA02C;">{org_conv}</b></div>
      </div>
        <div style="background:#0d1e2e; border-radius:10px; padding:9px
14px; margin-bottom:10px; font-size:0.82rem;">
           <span style="color:#aaa;">Suggested Contact Date: </span><b
style="color:{color};">{followup}</b>
      </div>
       <div style="background:{color}22; border-left:4px solid {color};
border-radius:0 8px 8px 0; padding:11px 14px; font-size:0.83rem; line-
height:1.7; color:#ddd;">
        <u>Sales Directive</u><br><br>{directive}
      </div>
    </div>"""


w_name = widgets.Text(value='', placeholder='e.g. Riya Sharma',
description='👤 Lead Name:', style={"description_width":"160px"},
layout=widgets.Layout(width="420px"))
sl_time = widgets.IntSlider(value=800, min=0, max=2500, step=50,
description="⏱️ Time on Web (s):", style={"description_width":"160px"},
layout=widgets.Layout(width="420px"))
sl_visits = widgets.IntSlider(value=3, min=0, max=30, description="👁
Total Visits:", style={"description_width":"160px"},
layout=widgets.Layout(width="420px"))
sl_pages = widgets.FloatSlider(value=2.0, min=0.0, max=10.0, step=0.5,
description="📄 Pages/Visit:", style={"description_width":"160px"},
layout=widgets.Layout(width="420px"), readout_format=".1f")
dd_occ = widgets.Dropdown(options=_OCCUPATIONS, value="Unemployed",
description="💼 Occupation:", style={"description_width":"160px"},
layout=widgets.Layout(width="420px"))
dd_src = widgets.Dropdown(options=_LEAD_SOURCES, value="Google",
description="📣 Lead Source:", style={"description_width":"160px"},
layout=widgets.Layout(width="420px"))
```

```
dd_act = widgets.Dropdown(options=_LAST_ACTIVITIES, value="Email Opened",
description="📋 Last Activity:", style={"description_width":"160px"},
layout=widgets.Layout(width="420px"))

dd_org = widgets.Dropdown(options=_LEAD_ORIGINS, value="Landing Page
Submission", description="🚀 Lead Origin:",
style={"description_width":"160px"},
layout=widgets.Layout(width="420px"))


out_score = widgets.Output()

out_db = widgets.Output()


btn_save = widgets.Button(description="Save Lead",
button_style="success", layout=widgets.Layout(width="155px",
height="38px"))

btn_db = widgets.Button(description="Pipeline View",
button_style="info", layout=widgets.Layout(width="165px",
height="38px"))

btn_export = widgets.Button(description="Export CRM CSV",
button_style="", layout=widgets.Layout(width="330px", height="38px"))


_state = {"db_open": False}


#
==========================================================================
======

# UPDATED _update FUNCTION: Includes error catching for IPyWidgets
debugging

#
==========================================================================
======

def _update(change=None):

    try:

        name = w_name.value

        tw, vis, pgv = sl_time.value, sl_visits.value, sl_pages.value

         occ, src, act, org = dd_occ.value, dd_src.value, dd_act.value,
dd_org.value
```

```
        row   = _build_input_row(tw, vis, pgv, occ, src, act, org)
        score = _score_lead(row)
        tier, emoji, color, directive = _tier_info(score)

        with out_score:
            clear_output(wait=True)
               display(HTML(_render_scorecard(score, tier, emoji, color,
directive, name, tw, vis, pgv, occ, src, act, org)))

    except Exception as e:
        with out_score:
            clear_output()
            print(f"CRITICAL ERROR IN SCORING LOGIC:\n{str(e)}")
#
==========================================================================
======

for _w in [w_name, sl_time, sl_visits, sl_pages, dd_occ, dd_src, dd_act,
dd_org]:
    _w.observe(_update, names="value")

def _on_save(b):
    tw, vis, pgv = sl_time.value, sl_visits.value, sl_pages.value
      occ, src, act, org = dd_occ.value, dd_src.value, dd_act.value,
dd_org.value
    name = w_name.value.strip() or "Unnamed Lead"

    row   = _build_input_row(tw, vis, pgv, occ, src, act, org)
    score = _score_lead(row)
    tier, emoji, *_ = _tier_info(score)
```

```
record = {
    "timestamp"           : time.strftime("%Y-%m-%d %H:%M:%S"),
    "Lead_Name"           : name,
    "Time_on_Web_s"       : tw,
    "Total_Visits"        : vis,
    "Page_Views_per_Visit": pgv,
    "Occupation"          : occ,
    "Lead_Source"         : src,
    "Last_Activity"       : act,
    "Lead_Origin"         : org,
    "Lead_Score"          : score,
    "Tier"                : f"{emoji} {tier}",
    "Followup_Date"       : _followup_date(tier)
}

existing = _load_session()
existing.append(record)
with open(_SESSION_FILE, "w") as f:
    json.dump(existing, f, indent=2)

btn_save.description  = "Saved!"
btn_save.button_style = "warning"
btn_save.disabled     = True

def _reset(change):
    btn_save.description  = "Save Lead"
    btn_save.button_style = "success"
    btn_save.unobserve(_reset, names="disabled")

btn_save.observe(_reset, names="disabled")
import threading
```

```
          threading.Timer(2.0,   lambda:   setattr(btn_save,   "disabled",
False)).start()


btn_save.on_click(_on_save)


def _kpi_card(label, value, color):

           return   f"<div   style='background:{color};color:#fff;border-
radius:10px;padding:12px    18px;text-align:center;min-width:90px;'><div
style='font-size:1.5rem;font-weight:900;'>{value}</div><div  style='font-
size:0.73rem;opacity:0.85;margin-top:2px;'>{label}</div></div>"


def _on_db_toggle(b):

    if _state["db_open"]:

        with out_db: clear_output()

       btn_db.description = "Pipeline View"; btn_db.button_style = "info"

        _state["db_open"] = False; return


    data = _load_session()

    if not data:

        with out_db:

            clear_output(wait=True)

                display(HTML("<p  style='color:#FF4136;font-family:Segoe
UI,Arial;'>No saved leads yet. Click Save Lead first.</p>"))

          btn_db.description  =  "Close  Pipeline";  btn_db.button_style  =
"danger"

        _state["db_open"] = True; return


    # Ensure no old revenue data makes it to the DataFrame

    cleaned_data = []

    for d in data:

        c = d.copy()

        c.pop("Expected_Revenue_INR", None)

        c.pop("Net_Expected_INR", None)
```

```
        cleaned_data.append(c)


    _df = pd.DataFrame(cleaned_data)


            if 'Lead_Score' in _df.columns: _df['Lead_Score'] =
_df['Lead_Score'].map('{:.1f}'.format)
    if 'Page_Views_per_Visit' in _df.columns: _df['Page_Views_per_Visit']
= _df['Page_Views_per_Visit'].map('{:.1f}'.format)


    tot = len(_df)
    avg_sc = pd.to_numeric(_df["Lead_Score"]).mean()
    def _cnt(kw): return len(_df[_df["Tier"].str.contains(kw, na=False)])
        hot_n, warm_n, cold_n, dead_n = _cnt("HOT"), _cnt("WARM"),
_cnt("COLD"), _cnt("DEAD")


    kpi_html = f"""<div style='font-family:Segoe UI,Arial;padding:10px;'>
                <h3 style='margin:0 0 12px;color:#1e293b;'>Sales Pipeline
Dashboard</h3>
                 <div style='display:flex;gap:10px;flex-wrap:wrap;margin-
bottom:16px;'>
                {_kpi_card('Total Leads', tot, '#334155')}
                {_kpi_card('Avg Score', f'{avg_sc:.1f}', '#1e40af')}
                {_kpi_card('HOT', hot_n, '#dc2626')}
                {_kpi_card('WARM', warm_n, '#ea580c')}
                {_kpi_card('COLD', cold_n, '#2563eb')}
                {_kpi_card('DEAD', dead_n, '#6b7280')}
                </div></div>"""


    _styled = (_df.style
               .set_table_styles([
                {"selector":"thead th","props":[("background","#1e293b"),
("color","#fff"),("padding","8px 12px"),("font-size","0.82rem")]},
```

```
                {"selector":"tbody td","props":[("padding","6px 12px"),
("font-size","0.81rem"),("color","#222"),("border-bottom","1px     solid
#e2e8f0"),("background","#ffffff")]},
                    {"selector":"tbody tr:nth-child(even) td","props":
[("background","#f8fafc")]},
                                {"selector":"table","props":[("border-
collapse","collapse"),("width","100%")]},
            ])
            .hide(axis="index"))

    with out_db:
        clear_output(wait=True)
        display(HTML(kpi_html))
        display(_styled)

    btn_db.description = "Close Pipeline"; btn_db.button_style = "danger"
    _state["db_open"] = True

btn_db.on_click(_on_db_toggle)

def _on_export(b):
    data = _load_session()
    if not data:
        btn_export.description = "No Data!"
        btn_export.button_style = "danger"
        import threading
        threading.Timer(2.0, lambda: (setattr(btn_export, "description",
"Export CRM CSV"), setattr(btn_export, "button_style", ""))).start()
        return

    cleaned_data = []
    for d in data:
```

```
        c = d.copy()
        c.pop("Expected_Revenue_INR", None)
        c.pop("Net_Expected_INR", None)
        cleaned_data.append(c)

    fname = f"crm_export_{time.strftime('%Y%m%d_%H%M%S')}.csv"
    pd.DataFrame(cleaned_data).to_csv(fname, index=False)

    btn_export.description = f"Saved: {fname}"
    btn_export.button_style = "success"
    import threading
      threading.Timer(3.0, lambda: (setattr(btn_export, "description", "Export CRM CSV"), setattr(btn_export, "button_style", ""))).start()

btn_export.on_click(_on_export)

_title = widgets.HTML(f"""
<div style="font-family:'Segoe UI',Arial; background:linear-gradient(90deg,#1a1a2e,#16213e); color:#fff; border-radius:12px; padding:18px 28px; margin-bottom:16px; border-left:6px solid #FF851B;">
      <h2 style="margin:0 0 4px; font-size:1.45rem; letter-spacing:1px;">Predictive Lead Scoring &amp; Decision Support System</h2>
  <p style="margin:0; font-size:0.87rem; color:#aaa;">X Education Pipeline  |  Champion: <b style="color:#FF851B;">{champion_name if 'champion_name' in globals() else 'Random Forest'}</b></p>
</div>""")

_sep = widgets.HTML("<hr style='border:1px solid #ddd;margin:8px 0;'>")

_left = widgets.VBox([
        widgets.HTML("<b style='font-size:0.88rem;color:#444;'>▶ LEAD INPUT</b>"),
    w_name, sl_time, sl_visits, sl_pages, _sep,
```

```
    widgets.HTML("<b style='font-size:0.82rem;color:#666;'>▶ CATEGORICAL SIGNALS</b>"),
    dd_occ, dd_src, dd_act, dd_org, _sep,
    widgets.HBox([btn_save, btn_db], layout=widgets.Layout(gap="10px")),
    btn_export,
], layout=widgets.Layout(padding="16px", border="1px solid #ddd", border_radius="12px", min_width="450px", max_width="470px"))

_main = widgets.HBox([_left, out_score], layout=widgets.Layout(flex_flow="row wrap", align_items="flex-start", gap="20px"))

_app = widgets.VBox([_title, _main, out_db], layout=widgets.Layout(padding="10px"))

_update()
display(_app)
```